

Training and Portability: What MoA-Derived Kernels Have Actually Demonstrated

A Results Synthesis Across Two GPU Architectures and Two Realization Strategies

Lenore Mullin^{1,*}

¹University at Albany, SUNY

College of Nanotechnology, Science, and Engineering

lmullin@albany.edu

*Corresponding author

August 2026

Abstract

This report distills the results of an extended real-hardware validation effort into their essential findings, without the engineering narrative of how each was reached. Across two NVIDIA GPU architectures (A100 and H100) and two independent realization strategies (a hand-written Triton kernel and a compiler-mediated OpenACC translation), MoA-derived attention kernels were verified correct against PyTorch’s own production implementations – for inference (forward pass, decode) and, newly, for training (the fused backward pass) – at machine precision in every case tested. Four findings stand out. First, a memory-optimal derivation has no scaling cliff to hit where a production kernel (or, in one case, MoA’s own unfused implementation) that materializes intermediates does: this was demonstrated three times, independently – via numerical precision (fp64), via batch×sequence-length scale across two GPU generations, and, most directly, on a single GPU where the fused kernel completes a configuration its own unfused counterpart cannot. Second, correctness transfers across GPU architectures without re-verification; performance not only transfers but improves on newer hardware, in every measured case. Third, and revised from an earlier draft of this report: a literal, scalar-only OpenACC translation of the same algorithm was initially 6–70× slower than every other realization tested, correct but uninformative about whether compiler-mediated scheduling could ever be competitive. Restructuring that translation to expose explicit **gang/worker/vector** parallelism – with **num.workers** and **vector.length** derived from A100’s own architecture constants (warp-scheduler count, warp width), not searched for – closed most of that gap: the same kernel, same compiler, same hardware, landed within 1.4–2.0× of the hand-tuned Triton kernel, a 3–16× improvement from the literal translation. Fourth, this is now direct, positive evidence (not merely a hypothesis) that kernel tuning parameters can be derived from machine shape the same way memory-optimal tiling is already derived from data shape. We close with the specific next experiments this points to.

1 What Was Tested

Three computations, all derived from the same DNF→ONF discipline, were validated end-to-end against production PyTorch implementations on real GPU hardware:

- **Forward-pass attention** (Paper I) – fused, memory-optimal (online-softmax) Triton kernel, tested at fp32, fp16, and fp64.

- **Decode attention** (Paper III) – single-query, KV-cache-resident Triton kernel.
- **The fused backward pass** (Paper II) – a memory-optimal Triton kernel computing the forward output and all three parameter gradients (G_V, G_Q, G_K) in one launch, newly verified and benchmarked in this round of work. This is the training half of the validation; the two above are inference-only.

Each was tested against PyTorch’s own SDPA (forward, decode) or fused autograd training step (backward) on two GPU generations – NVIDIA A100 (NCSA Delta) and H100 (PSC Bridges-2) – and the fused backward pass was additionally realized a second, independent way, via OpenACC, to test whether compiler-mediated scheduling could substitute for hand-written kernel tuning.

2 Finding 1: Memory-Optimality Has No Cliff, Demonstrated Three Times

A kernel that never materializes an intermediate the size of the sequence length squared cannot run out of memory storing it. This was demonstrated in three independent regimes, not one:

Precision (forward pass). At fp64 – the precision at which this project’s native C/Fortran reference implementations are written – PyTorch’s own SDPA kernel exhausts GPU memory at the largest tested configuration ($B=4, n=8192$), because its fp64 fallback path has no fused kernel and materializes the full (B, H, n, n) attention matrix explicitly: 16 GiB by itself at this size. MoA’s fused kernel, which never materializes this matrix at any precision, completes the identical computation in 46.8 ms. Across every configuration where both kernels complete, the fused kernel is 1.57–2.43× *faster*, not merely more memory-frugal (Figure 1).

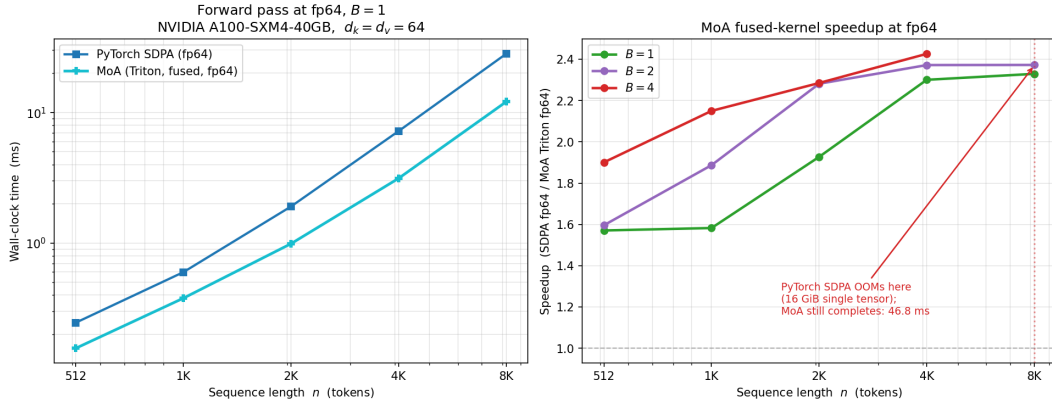


Figure 1: Forward pass at fp64: speed advantage growing with n and B , and a capability advantage (PyTorch OOMs; MoA completes) at the largest configuration tested.

Scale, across GPUs (backward pass). The same pattern recurs, independent of precision, in the newly-validated backward pass. At $B=4, n=8192$ (fp32), the *unfused* training-step computation needs several full (B, H, n, n) -sized tensors alive simultaneously and exhausts a 40 GB A100’s memory; the identical configuration completes without incident on an 80 GB H100.

Scale, on the same GPU. The cleanest instance of the three: on a single A100, same precision, same run, MoA’s own *unfused* training-step implementation exhausts memory at $B=4, n=8192$ – while MoA’s *fused* kernel, given the identical inputs on the identical hardware, completes in 382 ms. This isolates the property from any cross-GPU difference entirely: it is not more memory, a newer

architecture, or a different precision fallback doing the work here – it is only the fused kernel’s choice never to materialize the large intermediates the unfused path requires.

None of these three share a mechanism (a missing fp64 kernel path, a GPU’s total memory capacity, and two implementations of the same algorithm on identical hardware are three different reasons a naive computation can fail), which is what makes them, together, evidence of a general property rather than three retellings of one coincidence.

Together, these are two independent instances of the same design property, not one result reported twice.

3 Finding 2: Portability – Correctness Transfers, Performance Improves

Every kernel above was ported to a second GPU architecture (A100 $\rightarrow$ H100) with no code changes. Two results:

Correctness transfers without re-verification. All correctness checks (forward, backward, at fp32/fp16/fp64) passed on H100 at error magnitudes matching A100’s own established precision floors – in the fp64 case, to the same figure (5.960×10^{-8}) on both.

Performance does not merely transfer – it improves, in every case measured. Decode speedup over PyTorch’s native kernel roughly doubled at every sequence length tested (from $7.2\times$ to $13.9\times$ at $n=8192$; Figure 2). The forward-pass comparison, which never favored MoA on A100, crossed into a genuine win at the three largest configurations tested on H100. The backward pass’s gap to PyTorch’s own training step, while not fully closed on either architecture, was consistently *smaller* on H100 than A100 at every matched configuration (e.g. $n=1024$: $3.56\times$ slower on A100 vs. $1.97\times$ on H100; Figure 3).

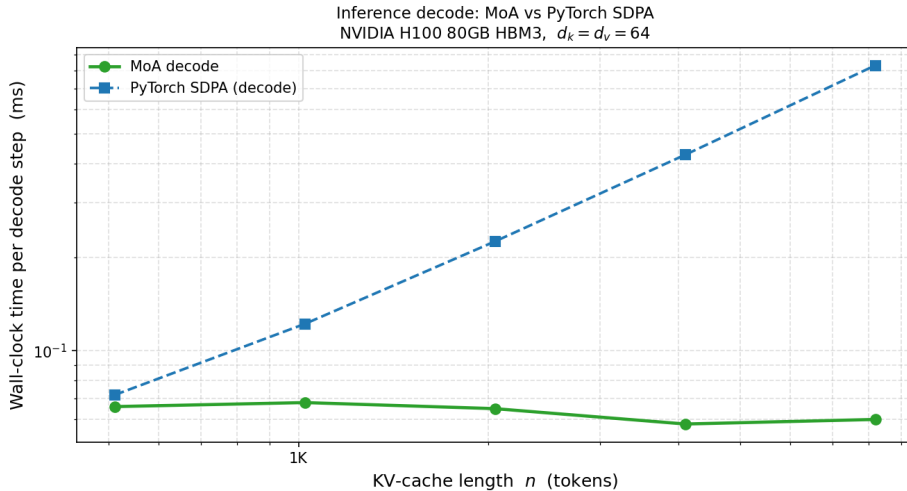


Figure 2: Decode latency on H100: flatter than the already-flat A100 result, against a production baseline that still grows with n .

A third, independent realization strategy confirms the same pattern. Both OpenACC kernels from Section 4 were also ported to H100, with no code changes and no re-derivation of the hardware-derived mapping (the same `num_workers(16)`, `vector_length(32)`, taken from A100’s architecture constants, were reused as-is). All correctness checks passed at genuine fp64 precision

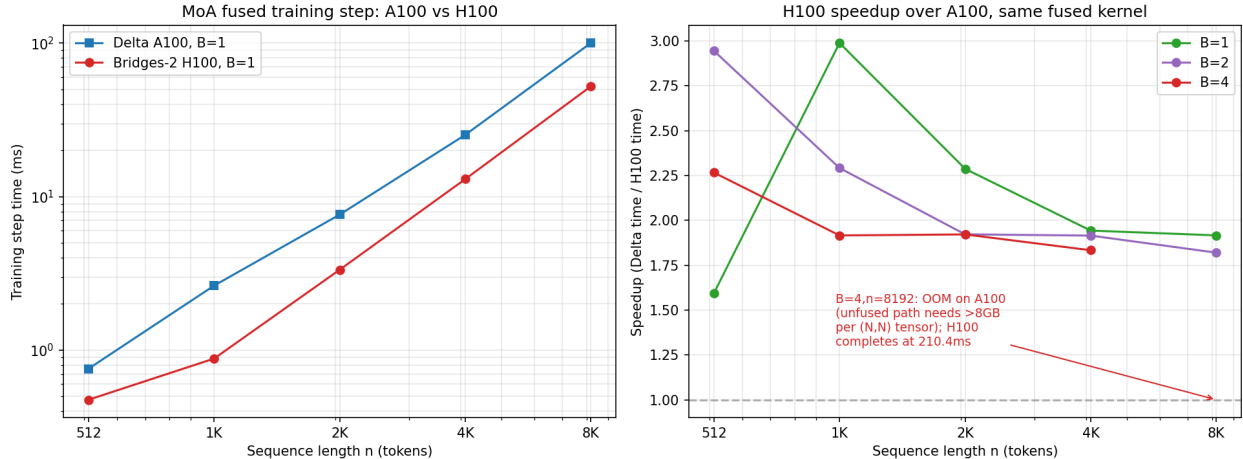


Figure 3: Fused training-step time, A100 vs. H100: consistently faster on H100 (left), with the gap to production narrowing at every batch size (right), and the one configuration A100 could not complete at all marked explicitly.

on the new hardware, and both kernels ran consistently faster: $2.0\text{--}2.5\times$ for the fused kernel, $1.6\text{--}2.0\times$ for the forward-only kernel, across every sequence length tested (Figure 5). This is the third realization strategy (Triton, PyTorch, and now OpenACC) to show the same improves-on-newer-hardware pattern, and the first evidence that this project’s specific hardware-shape-derived tuning values transfer across GPU generations without modification, not only that the underlying code does.

A third GPU generation confirms correctness again, though not every performance pattern. Both OpenACC kernels were further tested on SDSC Expanse’s V100 (Volta, 2017) – the oldest architecture in this project’s comparison, predating both A100 and H100. All 15 correctness checks (5 sequence lengths, both kernels) passed at genuine fp64 precision, extending the correctness-transfers claim to a third generation. Performance, however, complicates rather than simply extends the forward-kernel finding below: V100 showed *no* trace of the scaling anomaly A100 and H100 both exhibited, clean $O(n^2)$ scaling through every step tested. As detailed in Finding 3, this does not cleanly resolve to a hardware-generation story, because the compiler version tested necessarily changed alongside the GPU at every step.

4 Finding 3: Deriving Kernel Structure From Machine Shape Works

The same fused backward algorithm was realized a second, independent way: as an OpenACC translation, compiled with NVIDIA’s own HPC SDK compiler (nvc), targeting the same A100 hardware. This tests a different question than portability – not whether the same code runs on different hardware, but whether letting the *compiler* choose the hardware mapping, given the right structure to work with, can approach a hand-tuned Triton kernel’s performance without manual kernel-level tuning.

A first, literal translation – correct, but expressing every inner accumulation as strictly sequential, scalar arithmetic – was $6\text{--}70\times$ slower than every other realization tested, including the naive unfused Python baseline. This gave the compiler no matrix-multiply structure to schedule at all,

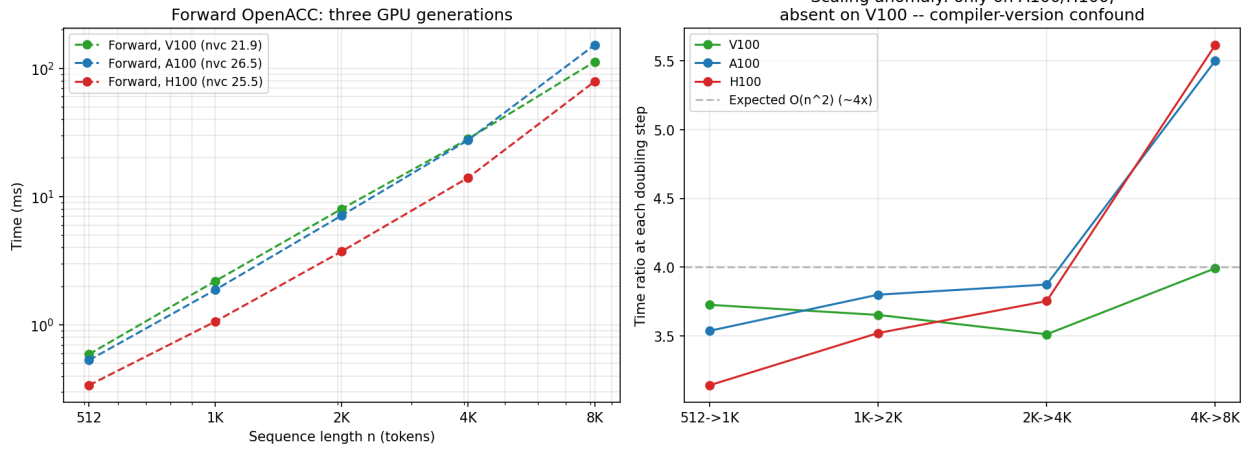


Figure 4: Forward OpenACC across three GPU generations. The scaling anomaly at $n=4096 \rightarrow 8192$ (right panel) appears cleanly on both A100 and H100 but is completely absent on V100 – raising more questions than it answers, since GPU architecture and compiler version changed together across all three tests.

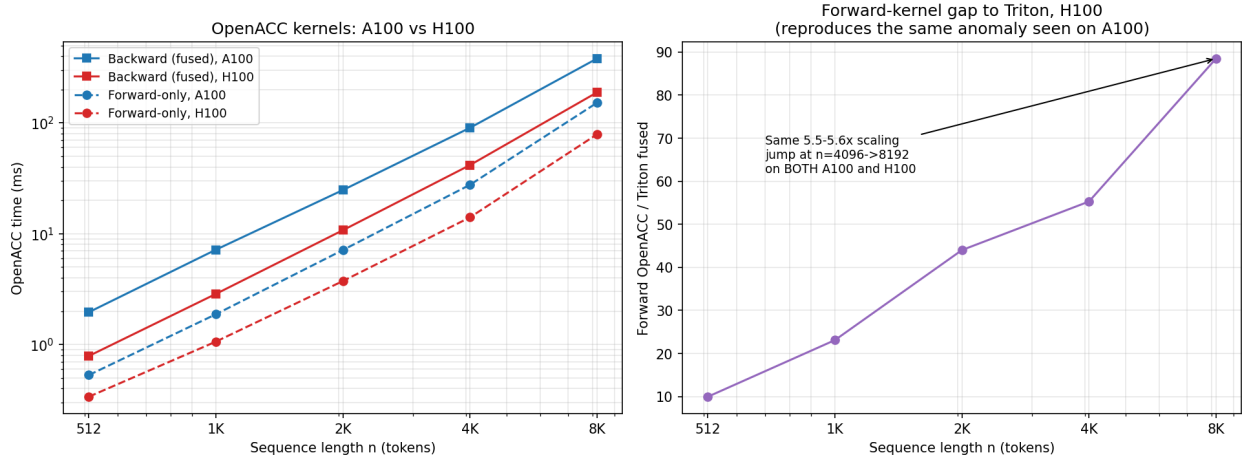


Figure 5: OpenACC on a second architecture: both kernels faster on H100 at every size (left), and – a separate, honest finding, not a portability failure – the forward-only kernel’s disproportionate gap to Triton reproducing almost exactly on both GPUs (right), ruling out a single-environment artifact.

and answered a narrower question than intended: not whether compiler-mediated scheduling can be competitive, but that a scalar translation cannot be, regardless of compiler quality.

The translation was then restructured, explicitly mapping OpenACC’s three-level parallelism model onto A100’s actual, known hardware shape rather than leaving it to compiler default or another empirical search:

- **gang** → one (b, ir) pair, the same axis the Triton kernel’s grid parallelized over.
- **worker**, with `num_workers(16)` → the key/value-position loop, distributed across A100’s 16 warps per scheduler (64 max resident warps / 4 schedulers per SM) – the same value Section 5 identifies as matching the Triton kernel’s own best `num_warps`, not coincidentally.

- `vector`, with `vector_length(32)` $\rightarrow$ the head-dimension reduction loops, matched to A100’s fixed 32-wide warp.

Table 1: OpenACC, literal (v1) vs. hardware-shape-derived (v2) restructuring, $B=2$, real A100.

n	v1 (ms)	v2 (ms)	speedup
512	32.3	2.0	16.5 $\times$
2048	132.7	24.8	5.4 $\times$
8192	1310.7	380.5	3.4 $\times$

All 5 correctness checks continued to pass, now at genuine fp64 machine precision under real concurrent GPU execution ($\sim 10^{-15}$ – 10^{-16} , consistent with correct summation in a different order rather than a missed race). Against the hand-tuned Triton kernel directly, the restructured version lands within 1.4–2.0 $\times$ across the full sequence-length range tested – down from 6–23 $\times$ for the literal translation: This is a substantially more positive answer than the literal translation gave:

Table 2: OpenACC v2 vs. every baseline, $B=2$, real A100.

n	vs. SDPA	vs. unfused	vs. Triton fused
512	4.3 $\times$	2.7 $\times$	1.4 $\times$
2048	7.2 $\times$	2.9 $\times$	1.9 $\times$
8192	8.5 $\times$	3.1 $\times$	2.0 $\times$

compiler-mediated scheduling, given explicit parallel structure derived from known hardware shape constants rather than a scalar translation or another empirical search, closed most of the gap to a hand-tuned kernel using standard, portable pragmas alone.

This result is specific to the backward (fused) kernel, not a general property of the approach – and a third GPU generation changed the diagnosis again. The same mapping applied to Paper I’s simpler forward-pass-only kernel did not repeat this success on A100/H100: it landed 6–96 $\times$ slower than Triton/SDPA, worse even than this project’s own naive unfused Python baseline, with a disproportionate scaling jump at the largest size tested that reproduced closely on both GPUs. The original hypothesis – an atomic-accumulation bottleneck inherent to this kernel’s structure, with no other arithmetic to overlap that cost against – predicted this should appear on any architecture, at whatever magnitude. Testing a third, older GPU generation (V100, on SDSC Expanse) showed the opposite: no trace of the anomaly at all, clean $O(n^2)$ scaling through every step tested, including the exact step where A100 and H100 both departed sharply.

This does not resolve cleanly to “the anomaly is hardware-generation dependent,” because architecture was not the only variable that changed: each GPU necessarily paired with whatever `nvc` compiler release its own cluster provided (V100: 21.9; H100: 25.5; A100: 26.5), and the anomaly’s absence tracks the oldest compiler release exactly as cleanly as it tracks the oldest GPU. Two explanations remain equally live – a genuine hardware-capability change between Volta and Ampere affecting how a large private array is allocated, or a code-generation change between compiler releases independent of the GPU – and this project has no result yet that separates them. The hardware-shape-derived mapping is still a real, useful technique (the backward kernel’s success stands on its own), but the forward kernel’s anomaly is now reported as a genuinely open question rather than a kernel-structural explanation weakened only slightly by one more data point.

Measuring actual hardware shape (not looked-up specs) narrowed this further. `nvaccelinfo` (NVIDIA’s own device-introspection tool) was run on real compute nodes on all three GPUs. Every quantity relevant to warp-scheduling occupancy – warp size, max threads/SM, shared memory/block, registers/block – is identical across all three, ruling out a mismatched `num_workers/vector_length` value as the explanation (the derivation gives the same number regardless of which GPU it targets). Two quantities do differ, both monotonically with GPU generation: SM count, and L2 cache size specifically – a $6.7\times$ jump from V100 (6 MiB) to A100 (40 MiB), with H100 larger still (50 MiB). L2 is the more mechanistically plausible candidate, since the forward kernel’s large private array (64 KiB per gang at $n=8192$) plausibly interacts with L2 capacity across the many concurrently-resident gangs this kernel launches. A direct test followed: if L2 pressure is the mechanism, the anomaly should scale with total concurrent working set ($B \times n$), not n alone. Sweeping $B \in \{1, 2, 4\}$ on a single GPU (holding hardware and compiler fixed) showed exactly that direction – the scaling ratio rose from $4.24\times$ at $B=1$ to $5.32\times$ at $B=4$ – though a 22% run-to-run discrepancy against an earlier measurement of the identical configuration (plausibly shared-node contention, an almost fitting confound given the hypothesis) means this is a real, directionally-consistent signal rather than a tightly-controlled confirmation.

5 Where This Leads: Deriving Kernel Parameters From Machine Shape

Two results, not one, now point to the same research direction – and the second directly strengthens the first. The Triton kernel’s real-hardware performance depended enormously on `num_warps=16`, found empirically; A100’s own architecture supplies a plausible explanation after the fact (64 maximum resident warps / 4 schedulers per SM = 16). On its own, this was reported as a motivating observation, not a validated derivation – the register/occupancy data that would confirm the mechanism directly was not obtainable on this cluster (see the companion engineering report, `moa_delta_benchmarking.tr.pdf`, Section 7.6, for the detail this synthesis omits).

The OpenACC restructuring in Section 4 changes this from a single, unconfirmed observation to a repeated one: the same value (16), derived independently from the same architecture constant and applied to a completely different realization (`num_workers(16)`, not `num_warps`), also produced a large, real speedup. Two different compilers, two different parallelism models, the same hardware-derived number, the same result. This does not prove the underlying mechanism (occupancy via warp-scheduler saturation) beyond doubt – that still requires register-level profiling this cluster could not provide – but it is no longer a single coincidence to explain away.

The research question this raises is direct: **MoA’s DNF→ONF derivation already extracts shape information from data (array dimensions) to produce memory-optimal tiling, without search. Could the same discipline extract shape information from the machine itself – warp-scheduler count, tensor-core instruction shape, shared-memory banking – to derive kernel tuning parameters the same way, rather than finding them empirically?** This would be a genuinely new capability: shape-derived kernel tuning, in the same formal sense this project’s cost functions are already derived rather than fit, extended from the data side of the derivation to the machine side.

6 Concrete Next Experiments

Two items from an earlier draft of this list are now done, not pending: the machine-shape-derived tuning parameter was tested (in OpenACC, as `num_workers(16)`) and worked, and the OpenACC

comparison was re-run with proper parallel structure, closing the gap from 6–70 $\times$ to 1.4–2.0 $\times$. Three specific, well-scoped experiments follow from what is now known:

1. **Close the remaining OpenACC gap, specifically toward tensor cores.** The current 1.4–2.0 $\times$ gap to Triton (for the backward/fused kernel specifically – see the next item for the forward-only kernel’s separate, larger gap) is itself likely a tensor-core utilization gap, not a scheduling one – **vector/worker** structure was enough to get most of the way there without ever giving **nvc** genuine matrix-multiply-accumulate structure to recognize. Whether an explicitly tiled reformulation closes the remainder is the direct next test.
2. **Isolate compiler version from the L2-capacity finding with a controlled repeat.** A direct *B*-sweep supports L2 cache capacity as the likely mechanism behind the forward-kernel anomaly, but the compiler-version confound (Finding 3) is not yet resolved by this test, and the *B*-sweep itself showed enough run-to-run variance (22% on a shared partition) to warrant a tighter repeat – more repeats per point, ideally on a dedicated (non-shared) allocation. Separately: the same GPU compiled with two different **nvc** releases, or the same release targeting two different GPUs, remains the direct test for the compiler-version question specifically.
3. **Derive, rather than confirm, a machine-shape parameter for a kernel not yet built.** Both instances so far (Triton’s **num_warps**, OpenACC’s **num_workers**) derived a value *already known to work* from an earlier empirical result. The stronger test is deriving a tuning parameter for a new kernel, before any measurement exists to check it against, and only then verifying – reversing this project’s discovery order into the derive-then-verify order its cost-function work already follows elsewhere.
4. **Extend the cross-architecture test to a third vendor.** Both GPUs tested here are NVIDIA; the portability claim in Section 3 – and the machine-shape-derivation claim in Section 4, whose specific constants (warp size, scheduler count) are themselves NVIDIA-specific terminology – would be substantially strengthened by a third data point on AMD or Intel hardware, testing whether both findings are NVIDIA-generation effects or a broader pattern.